

LAPORAN PRATIKUM PEKAN 4 STRUKTUR DATA

“Queue”



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Avero

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas

2026

KATA PENGANTAR

Puji dan syukur senantiasa penulis panjatkan kepada Allah SWT yang telah melimpahkan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan laporan ini guna memenuhi laporan pratikum Mata Kuliah Praktikum Struktur Data, dengan judul: “Queue”. Laporan ini berisikan hasil analisis dari praktikum basis data penulis di kelas DIF62111/IF/Prakt/C yang dilaksanakan pada hari Selasa, tanggal 27 April 2026.

Pada kesempatan ini, Penulis ingin mengucapkan terima kasih kepada Bapak Ir Wahyudi, S.T, M.T, dosen pengampu Mata Kuliah Praktikum Struktur Data, yang telah memberikan tugas ini. Penulis juga ingin mengucapkan terima kasih kepada pihak-pihak yang telah mendukung serta membantu penulis dalam penyelesaian laporan pratikum Struktur Data.

Penulis sadar bahwa laporan ini masih memiliki beberapa kekurangan dikarenakan terbatasnya pengalaman dan pengetahuan yang penulis miliki. Oleh karena itu, penulis harap adanya bentuk saran dan kritik yang membangun dari berbagai pihak.

Akhir kata, penulis berharap laporan ini dapat bermanfaat bagi perkembangan dunia pendidikan.

Padang, 29 April 2026

Hamdi Sidqi Alifi

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Pratikum.....	1
BAB II PEMBAHASAN	2
2.1 QueueArray & QueueArrayDriver.....	2
2.2 ReverseData	5
2.3 IterasiQueue	6
2.4 QueueLinkedList.....	7
BAB III KESIMPULAN.....	8
4.1 Kesimpulan	8
4.2 Saran.....	8
BAB IV DAFTAR PUSTAKA	9
Lampiran Tugas	11
1. Deskripsi Tugas.....	11
2. Aturan Penamaan	11
3. Struktur Program.....	11
4. Contoh Tampilan Output Program.....	12
5. Kriteria Penilaian	Error! Bookmark not defined.
6. Deadline	13
Java (ADT) Class · Website_2511531009.....	14
Java (Main/Driver) Class · Browser_2511531009.....	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Secara bahasa, kita mengetahui Queue adalah berupa suatu antrian. Dalam Java queue merupakan data struktur yang mengumpulkan data yang mempunyai karakteristik “yang pertama masuk, keluar pertama”. Operasi dasar di Queue antara lain ialah enqueue, dequeue, peek, isEmpty, dan size.

Queue dapat diimplementasikan dalam bentuk array dan linkedlist. Sebagai contoh, Queue dapat diimplementasikan pada penjadwalan cetak pada mesin cetak di kantor, proses pemesanan tiket secara daring, atau bahkan seperti menciptakan algoritma untuk breath-first search pada pada grafik.

Queue sering kali dikaitkan dengan Stack, yang mana memiliki struktur yang hampir mirip.

1.2 Tujuan

1. Menenal Queue dan memahami karakteristiknya, khususnya prinsip FIFO (*First In, First Out*).
2. Menganalisis file Java yang mengimplementasikan struktur data Queue.

1.3 Manfaat Pratikum

1. Mampu menggunakan class Queue yang telah dipanggil ke dalam sebuah class Java.
2. Mampu menemukan dan memahami alur logika pada file Java yang memiliki class struktur data Queue.

BAB II PEMBAHASAN

2.1 QueueArray & QueueArrayDriver

```
1 package pekan4_2511531009;
2
3 public class QueueArray_2511531009 {
4     int front_1009, rear_1009, size_1009;
5     int capacity_1009;
6     int array_1009[];
7
8     // mengontruksikan queuearray
9     public QueueArray_2511531009(int capacity_1009) {
10         this.capacity_1009 = capacity_1009; // meng-assign capacity input ke capacity pada cl
11         front_1009 = this.size_1009 = 0;
12         rear_1009 = capacity_1009 - 1;
13         array_1009 = new int[this.capacity_1009];
14     }
15
16     boolean isFull_1009(QueueArray_2511531009 queue_1009) {
17         return (queue_1009.size_1009 == queue_1009.capacity_1009);
18     }
19
20     boolean isEmpty_1009(QueueArray_2511531009 queue_1009) {
21         return (queue_1009.size_1009 == 0);
22     }
23
24     void enqueue_1009(int item_1009) {
25         if (isFull_1009(this))
26             return;
27         this.rear_1009 = (this.rear_1009 + 1) % this.capacity_1009;
28         this.array_1009[this.rear_1009] = item_1009;
29         this.size_1009 = this.size_1009 + 1;
30         System.out.println(item_1009 + " enqueued to queue_1009"); // memberitahu user bahwa
31     }
32
33     int dequeue_1009() {
34         if (isEmpty_1009(this))
35             return Integer.MIN_VALUE;
36         int item_1009 = this.array_1009[this.front_1009];
37         this.front_1009 = (this.front_1009 + 1) % this.capacity_1009;
38         this.size_1009 = this.size_1009 - 1;
39         return item_1009;
40     }
41 }
```

Cuplikan kode di atas merupakan implementasi Queue menggunakan Array. Di dalam memory, elemen array tersimpan tepat setelah elemen sebelumnya.

Komputer hanya akan memiliki alamat untuk byte pertama dari array yang telah dibuat, jadi untuk mengakses elemen ketiga pada arr[2]. Saat menghapus atau mensisipkan data di dalam sebuah array, tiap-tiap elemen akan bergeser, entah bergeser keatas untuk membuat sebuah tempat bagi elemen baru, atau bergeser kebawah untuk mengambil posisi elemen yang telah dihapus. Memanipulasi Array sama halnya dengan memanupulasi data pada memori komputer.

Method pada line 16 berguna untuk memeriksa apakah queue full. Line 20 memeriksa apakah queue kosong

Line 24 memasukkan data

Line 33 mengeluarkan data yang berada di kepala

```
42●   int front_1009() {
43       if (isEmpty_1009(this))
44           return Integer.MIN_VALUE; // mengindikasikan bahwa queue berada pada kondisi koso
45
46       return this.array_1009[this.front_1009];
47   }
48
49●   int rear_1009() {
50       if (isEmpty_1009(this))return Integer.MIN_VALUE;
51
52       return this.array_1009[this.rear_1009];
53   }
54
55   // print queue element
56●   void display_1009() {
57       int i_1009;
58       if (front_1009 == rear_1009) {
59           System.out.printf("\nAntrian Kosong\n");
60           return;
61       }
62       // visit from last and print
63       for (i_1009 = front_1009; i_1009 < rear_1009; i_1009++) {
64           System.out.printf(" %d <-- ", array_1009[i_1009]);
65       }
66   }
67
68 }
```

Mereturn nilai depan,belakang, pada queue array

Display → untuk menampilkan array

```
1 package pekan4_2511531009;
2
3 public class QueueArrayDriver_2511531009 {
4
5●   public static void main(String[] args) {
6       QueueArray_2511531009 queue_1009 = new QueueArray_2511531009(1000);
7       queue_1009.enqueue_1009(10);
8       queue_1009.enqueue_1009(20);
9       queue_1009.enqueue_1009(30);
10      queue_1009.enqueue_1009(40);
11
12      System.out.println("Item di depan " + queue_1009.front_1009());
13      System.out.println("Item paling belakang " + queue_1009.rear_1009());
14
15      System.out.println("tampilan queue");
16      queue_1009.display_1009();
17      System.out.println();
18      System.out.println(queue_1009.dequeue_1009() + " dihapus dari queue");
19      System.out.println("Item di depan " + queue_1009.front_1009());
20      System.out.println("Item di belakang " + queue_1009.rear_1009());
21      System.out.println("tampilan queue setelah satu data dihapus");
22      queue_1009.display_1009();
23
24
25   }
26
27 }
```

```
10 enqueued to queue_1009
20 enqueued to queue_1009
30 enqueued to queue_1009
40 enqueued to queue_1009
Item di depan 10
Item paling belakang 40
tampilan queue
10 <-- 20 <-- 30 <--
10 dihapus dari queue
Item di depan 20
Item di belakang 40
tampilan queue setelah satu data dihapus
20 <-- 30 <--
```

2.2 ReverseData

```
1 package pekan4_2511531009;
2 import java.util.*;
3 public class ReverseData_2511531009 {
4
5     public static void main(String[] args) {
6         Queue<Integer> q_1009 = new LinkedList<Integer>();
7         q_1009.add(1);
8         q_1009.add(2);
9         q_1009.add(3); // [1, 2, 3]
10
11         System.out.println("sebelum reverse " + q_1009);
12         Stack<Integer> s_1009 = new Stack<Integer>();
13         while (!q_1009.isEmpty()) { // Q -> S
14             s_1009.push(q_1009.remove());
15         }
16
17         while (!s_1009.isEmpty()) { // S -> Q
18             q_1009.add(s_1009.pop());
19         }
20
21         System.out.println("sebelum reverse " + q_1009);
22
23     }
24
25 }
```

Reverse data merupakan gabungan dari data stack dengan queue yang menggunakan linkedlist sebagai implementasinya.

Pada awalnya data akan diassignkan pada queue, kemudian dijasikan stack, dan dikembalikan lagi menjadi queue setelah urutan dibalikkan

```
sebelum reverse [1, 2, 3]
sebelum reverse [3, 2, 1]
```

2.3 IterasiQueue

```
1 package pekan4_2511531009;
2 import java.util.*;
3 public class IterasiQueue_2511531009 {
4
5     public static void main(String[] args) {
6         Queue<String> q_1009 = new LinkedList<>();
7
8         q_1009.add("Praktikum");
9         q_1009.add("Struktur");
10        q_1009.add("Data");
11        q_1009.add("dan");
12        q_1009.add("Algoritma");
13
14        Iterator<String> ite_1009 = q_1009.iterator();
15        while (ite_1009.hasNext()) {
16            System.out.print(ite_1009.next() + " ");
17        }
18    }
19
20 }
```

Kode program ini berguna untuk menambahkan iterasi dengan queue melalui linked list

```
Praktikum Struktur Data dan Algoritma
```

2.4 QueueLinkedList

```
1 package pekan4_2511531009;
2 import java.util.*;
3 public class QueueLinkedList_2511531009 {
4
5     public static void main(String[] args) {
6         Queue<Integer> q_1009 = new LinkedList<Integer>();
7         // tambah elemen
8         for (int i_1009 = 0; i_1009 < 6; i_1009++)
9             q_1009.add(i_1009);
10
11         // menampilkan isi antrian
12         System.out.println("Elemen Antrian " + q_1009);
13
14         // menghapus kepala antrian
15         int hapus_1009 = q_1009.remove();
16         System.out.println("hapus elemen " + hapus_1009);
17         System.out.println(q_1009);
18
19         // melihat antrian terdepan
20         int depan_1009 = q_1009.peek();
21         System.out.println("Kepala antrian " + depan_1009);
22
23         // melihat size antrian
24         int banyak_1009 = q_1009.size();
25         System.out.println("Size antrian " + banyak_1009);
26     }
27 }
```

```
Elemen Antrian [0, 1, 2, 3, 4, 5]
hapus elemen 0
[1, 2, 3, 4, 5]
Kepala antrian 1
Size antrian 5
```

Pada file ini kita akan mempelajari implementasi Queue menggunakan LinkedList sebagai struktur penyimpanan datanya. Berbeda dengan implementasi berbasis array yang memiliki ukuran tetap, implementasi Queue melalui LinkedList memungkinkan ukuran yang dinamis karena setiap node hanya mengalokasikan memori sesuai kebutuhan tanpa perlu menggeser elemen lain saat terjadi proses enqueue maupun dequeue.

Pada LinkedList, setiap node menyimpan data sekaligus referensi (pointer) menuju node berikutnya. Sehingga operasi enqueue cukup dilakukan dengan menambahkan node baru di bagian ekor (rear) list, sedangkan operasi dequeue dilakukan dengan menghapus node yang berada di bagian kepala (front) list. Tidak seperti implementasi array, pada LinkedList kita tidak perlu khawatir kondisi queue penuh karena kapasitasnya bersifat fleksibel dan hanya dibatasi oleh memori yang tersedia pada sistem.

Pada kode tersebut, terdapat method enqueue() yang menambahkan elemen baru ke bagian belakang antrian, method dequeue() yang mengeluarkan dan mengembalikan elemen paling depan, serta method peek() untuk melihat elemen terdepan tanpa menghapusnya. Method isEmpty() digunakan untuk memeriksa apakah queue dalam kondisi kosong sebelum melakukan operasi dequeue agar tidak terjadi error saat antrian sudah habis. Output dari kode ini menunjukkan urutan elemen yang sesuai dengan prinsip FIFO, dimana elemen yang pertama di-enqueue akan menjadi elemen yang pertama kali di-dequeue.

BAB III KESIMPULAN

4.1 Kesimpulan

Beberapa alasan untuk mengimplementasikan Queue dengan Array ialah dengan tujuan untuk mencapai memori efisiensi dan pengaplikasian yang lebih mudah untuk dimengerti. Elemen Array tidak memiliki alamat untuk elemen selanjutnya seperti nodes pada linked list. Menggunakan Array untuk mengimplementasikan Queue memerlukan kodingan yang lebih sedikit dari pada linked list.

Namun pada beberapa kondisi, terdapat alasan untuk mengimplementasikan Queue dengan Array. Yang pertama ialah ukurannya yang tidak fleksible. Sebuah array mengakusisi beberapa bagian dari memory, dengan kata lain queuearray akan mengambil memori yang lebih banyak daripada biasanya. Dequeue menyebabkan elemen pertama untuk dihapus. Elemen yang lain akan bergeser untuk mengambil posisi elemn yang telah dihapus sebelumnya.

4.2 Saran

Seperti, mahasiswa sebaiknya diarahkan untuk lebih memahami perbedaan antara implementasi Queue menggunakan Array dengan implementasi menggunakan LinkedList, sehingga mereka dapat menentukan pendekatan yang paling sesuai berdasarkan kebutuhan kapasitas dan efisiensi program yang akan dibuat.

BAB IV

DAFTAR PUSTAKA

- [1] Laboratorium Pembelajaran, "Modul 07: ADT Queue," *Praktikum Algoritma dan Struktur Data*, Program Studi Teknik Komputer, Fakultas Ilmu Komputer, Universitas Brawijaya, Malang, Indonesia.
- [2] W3Schools, "DSA Queues," *W3Schools*. [Daring]. Tersedia pada: https://www.w3schools.com/dsa/dsa_data_queues.php. [Diakses: 29 Apr. 2026].
- [3] W3Schools, "DSA Linked Lists in Memory," *W3Schools*. [Daring]. Tersedia pada: https://www.w3schools.com/dsa/dsa_theory_linkedlists_memory.php. [Diakses: 29 Apr. 2026].

LAPORAN TUGAS PEKAN 4 STRUKTUR DATA



Dosen Pengampu:
Ph.D. Ir. Wahyudi, S.T., M.T.

Asisten Praktikum :
Muhammad Galid Averro

Disusun Oleh:
Hamdi Sidqi Alifi
2511531009

Fakultas Teknologi Informasi
Departemen Informatika
Universitas Andalas
2026

Lampiran Tugas

Topik : Implementasi Struktur Data Queue

1. Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan struktur data Queue (Antrian) menggunakan array dalam bahasa Java

Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip FIFO

Program harus mampu:

- Menambahkan pelanggan ke antrian (enqueue)
- Menghapus pelanggan dari antrian (dequeue)
- Menampilkan isi antrian
- Mengecek kondisi antrian (kosong atau penuh)

2. Aturan Penamaan

- Nama Kelas : concat sufiks “_” + NIM
Contoh : AntrianLoket_2511531009 dan Queue_2511531009
- Nama Variabel Atribut : concat sufiks “_” + NIM mod 10^4
Contoh : int front_1009, int rear_1009, int max_1009
- Nama File : Harus sama dengan nama class

3. Struktur Tugas

- Deklarasi dan Inisialisasi
 - Array untuk menyimpan data antrian
 - Variabel front_1009 dan rear_1009
 - Kapasitas maksimal queue

- Method yang Wajib Ada

-	enqueue()	:	Menampilkan data ke belakang antrian Jika penuh → tampilkan pesan
-	dequeue()	:	Menghapus data dari depan antrian Jika kosong → tampilkan pesan
-	display()	:	Menampilkan seluruh isi antrian
-	isEmpty()	:	Mengembalikan nilai true jika kosong
-	isFull()	:	Mengembalikan nilai true jika penuh
-	Reverse()	:	Membalik seluruh isi antrian

- c) Menu Program
== PROGRAM ANTRIAN LOKET ==
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar

4. Contoh Output

== PROGRAM ANTRIAN LOKET ==
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih Menu :

Masukkan nama pelanggan : Andi
Data berhasil ditambahkan ke antrian

Pilih Menu : 1
Masukkan Nama Pelanggan : Budi
Data berhasil ditambahkan ke antrian

Pilih Menu : 3
Isi Antrian :
1. Andi
2. Budi

Pilih Menu : 4
Isi Antrian :
1. Budi
2. Andi

Pilih Menu : 2
Budi telah dilayani

Pilih Menu :
Isi Antrian :
1. Andi

3. Deadline

Tugas dikumpulkan paling lambat pada hari Minggu, tanggal 29 April 2026, pukul 23.59 WIB.

Java (ADT) Class · Queue 2511531009

```
1 package pekan4_2511531009;
2
3 import java.util.Stack;
4
5 public class Queue_2511531009 {
6     int front_1009, rear_1009, size_1009;
7     int capacity_1009;
8     String[] array_1009;
9
10    // Konstruktor: inisialisasi antrian dengan kapasitas tertentu
11    public Queue_2511531009(int capacity_1009) {
12        this.capacity_1009 = capacity_1009;
13        front_1009 = 0;
14        rear_1009 = capacity_1009 - 1;
15        size_1009 = 0;
16        array_1009 = new String[this.capacity_1009];
17    }
18
19    // Cek apakah antrian penuh
20    boolean isFull_1009() {
21        return size_1009 == capacity_1009;
22    }
23
24    // Cek apakah antrian kosong
25    boolean isEmpty_1009() {
26        return size_1009 == 0;
27    }
28
29    // Menambahkan pelanggan ke belakang antrian (dengan pesan untuk pengguna)
30    void enqueue_1009(String item_1009) {
31        if (isFull_1009()) {
32            System.out.println("Antrian penuh! Tidak bisa menambahkan pelanggan.");
33            return;
34        }
35        rear_1009 = (rear_1009 + 1) % capacity_1009;
36        array_1009[rear_1009] = item_1009;
37        size_1009++;
38        System.out.println(item_1009 + " berhasil ditambahkan ke antrian.");
39    }
}
```

```

// Menghapus pelanggan dari depan antrian (dengan pesan untuk pengguna)
String dequeue_1009() {
    if (isEmpty_1009()) {
        System.out.println("Antrian kosong! Tidak ada pelanggan yang bisa dihapus.");
        return null;
    }
    String item_1009 = array_1009[front_1009];
    front_1009 = (front_1009 + 1) % capacity_1009;
    size_1009--;
    System.out.println(item_1009 + " telah dilayani dan dihapus dari antrian.");
    return item_1009;
}

// reverse
private void enqueueSilent_1009(String item_1009) {
    rear_1009 = (rear_1009 + 1) % capacity_1009;
    array_1009[rear_1009] = item_1009;
    size_1009++;
}

private String dequeueSilent_1009() {
    String item_1009 = array_1009[front_1009];
    front_1009 = (front_1009 + 1) % capacity_1009;
    size_1009--;
    return item_1009;
}

// Menampilkan seluruh isi antrian
void display_1009() {
    if (isEmpty_1009()) {
        System.out.println("Antrian kosong!");
        return;
    }
    System.out.println("Isi Antrian:");
    int idx = front_1009;
    for (int i = 0; i < size_1009; i++) {
        System.out.println((i + 1) + ". " + array_1009[idx]);
        idx = (idx + 1) % capacity_1009;
    }
}

```

```

82 // Membalik isi antrian
83 // 1. Pindahkan semua elemen dari queue ke stack (gunakan dequeueSilent)
84 // 2. Pindahkan semua elemen dari stack ke queue (gunakan enqueueSilent)
85 void reverse_1009() {
86     if (isEmpty_1009()) {
87         System.out.println("Antrian kosong! Tidak bisa dibalik.");
88         return;
89     }
90
91     Stack<String> stack_1009 = new Stack<>();
92
93     // Langkah 1: Queue → Stack
94     while (!isEmpty_1009()) {
95         stack_1009.push(dequeueSilent_1009());
96     }
97
98     // Langkah 2: Stack → Queue
99     while (!stack_1009.isEmpty()) {
100         enqueueSilent_1009(stack_1009.pop());
101     }
102
103     System.out.println("Antrian berhasil dibalik.");
104 }
105 }

```

Java (Main/Driver) Class · AntrianLoket_2511531009

```
1 package pekan4_2511531009;
2
3 import java.util.Scanner;
4
5 public class AntrianLoket_2511531009 {
6     public static void main(String[] args) {
7         Scanner scanner_1009 = new Scanner(System.in);
8         Queue_2511531009 queue_1009 = new Queue_2511531009(10); // Kapasitas maksimal antrian adalah 10
9
10        int pilihan_1009;
11        do {
12            System.out.println("\n== PROGRAM ANTRIAN LOKET ==");
13            System.out.println("1. Tambah Antrian");
14            System.out.println("2. Hapus Antrian");
15            System.out.println("3. Tampilkan Antrian");
16            System.out.println("4. Reverse");
17            System.out.println("5. Keluar");
18            System.out.print("Pilih Menu : ");
19            pilihan_1009 = scanner_1009.nextInt();
20            scanner_1009.nextLine(); // Membersihkan newline
21
22            switch (pilihan_1009) {
23                case 1:
24                    System.out.print("Masukkan nama pelanggan : ");
25                    String namaPelanggan_1009 = scanner_1009.nextLine();
26                    queue_1009.enqueue_1009(namaPelanggan_1009);
27                    break;
28                case 2:
29                    queue_1009.dequeue_1009();
30                    break;
31                case 3:
32                    queue_1009.display_1009();
33                    break;
34                case 4:
35                    queue_1009.reverse_1009();
36                    break;
37                case 5:
38                    System.out.println("Keluar dari program...");
39                    break;
40                default:
41                    System.out.println("Menu tidak valid! Silakan coba lagi.");
42            }
43        } while (pilihan_1009 != 5);
44        scanner_1009.close();
45    }
46 }
47 }
```

```
== PROGRAM ANTRIAN LOKET ==  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih Menu : 1  
Masukkan nama pelanggan : James  
James berhasil ditambahkan ke antrian.  
  
== PROGRAM ANTRIAN LOKET ==  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih Menu : 1  
Masukkan nama pelanggan : Burton  
Burton berhasil ditambahkan ke antrian.  
  
== PROGRAM ANTRIAN LOKET ==  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih Menu : 3  
Isi Antrian:  
1. James  
2. Burton  
  
== PROGRAM ANTRIAN LOKET ==  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih Menu : 4  
Antrian berhasil dibalik.
```

```
== PROGRAM ANTRIAN LOKET ==  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih Menu : 3  
Isi Antrian:  
1. Burton  
2. James  
  
== PROGRAM ANTRIAN LOKET ==  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih Menu : 2  
Burton telah dilayani dan dihapus dari antr  
  
== PROGRAM ANTRIAN LOKET ==  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih Menu : 3  
Isi Antrian:  
1. James
```

```
== PROGRAM ANTRIAN LOKET ==  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih Menu : 6  
Menu tidak valid! Silakan coba lagi.  
  
== PROGRAM ANTRIAN LOKET ==  
1. Tambah Antrian  
2. Hapus Antrian  
3. Tampilkan Antrian  
4. Reverse  
5. Keluar  
Pilih Menu : 5  
Keluar dari program...
```